

Phase 2 Implementation and Progress Report

CPU Scheduling Simulator

Operating Systems Project

April 25, 2026

1 Phase 2 Requirement Checklist

Phase 2 requires movement from design into working implementation. This submission satisfies the core requirement by providing a functional C implementation of the main CPU scheduling modules, test coverage for the core logic, an organized source tree, executable examples, a browser dashboard, and this progress report.

Requirement	Status	Evidence in Repository
Partial implementation of core features	Implemented	Four scheduling algorithms, workload parsing, simulation timeline generation, metric calculation, comparison mode, CSV export, tests, and dashboard are present.
At least 50–70% implemented	Implemented	The project is approximately 70% complete. The core simulator is working; remaining work is final validation, polish, and report/PDF packaging.
Progress report as PDF	Implemented	The report source is <code>reports/phase2_report.tex</code> . The compiled artifact is <code>reports/phase2_report.pdf</code> .
Brief recap of project idea	Implemented	Section 2 summarizes the project goal and scope.
What has been implemented so far	Implemented	Section 3 lists implemented components and their current status.
Updated system architecture	Implemented	Section 4 includes the current architecture and module responsibilities.
Screenshots of working system	Implemented	Section 5 includes terminal evidence and dashboard screenshot references.
Challenges faced	Implemented	Section 6 documents implementation and validation challenges.
Plan for completing remaining work	Implemented	Section 7 lists the final project plan.

Code submission	Implemented	Section 8 describes the organized source structure and key files.
Proper folder structure	Implemented	Source, headers, tests, workloads, scripts, dashboard, fuzzing, and reports are separated.
Comments in code	Implemented	Public headers and selected implementation sections include comments where behavior is non-obvious.

2 Project Recap

The project is a CPU scheduling simulator for an Operating Systems course. It models a set of processes with arrival time, burst time, and priority, then runs scheduling algorithms to show how process order affects waiting time, turnaround time, response time, CPU utilization, throughput, and context switching.

The implemented simulator supports:

- First-Come, First-Served (FCFS)
- non-preemptive Shortest Job First (SJF)
- Round Robin with configurable quantum
- non-preemptive priority scheduling

The project is intentionally focused on operating-system scheduling behavior. Earlier experimental prediction and cloud inference files were removed during final Phase 2 cleanup so the repository now matches the scheduler-only project scope.

3 Implemented So Far

The current implementation is approximately 70% complete. The core simulator is working and organized, but final submission still needs stronger validation evidence such as Valgrind or sanitizer logs, longer fuzzing results, and final PDF packaging.

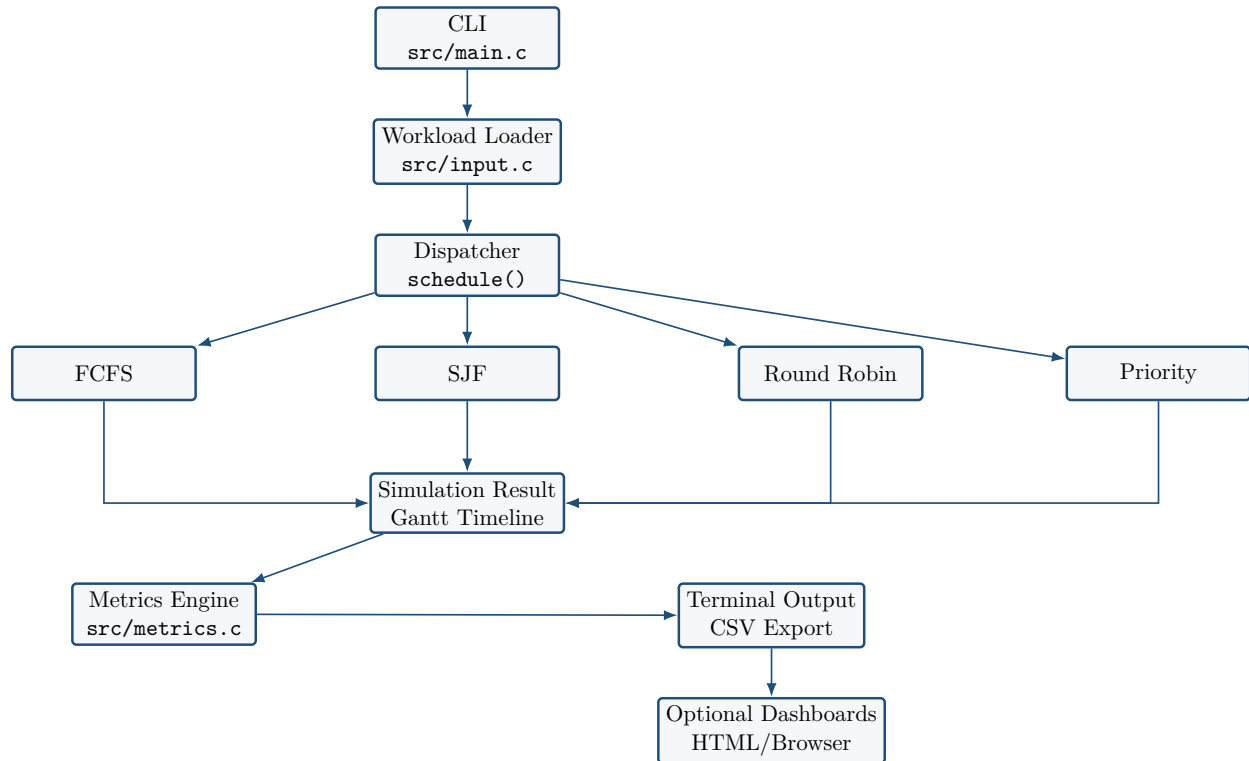
Component	Status	Implemented Behavior
Process model	Implemented	Process stores PID, arrival time, burst time, priority, remaining time, state, start time, completion time, waiting time, turnaround time, and response time.
Process reset	Implemented	Before every algorithm run, simulation-only fields are reset so comparison mode does not reuse stale timing state.

Input parser	Implemented	Loads workload files and in-memory workload buffers. It accepts comments and blank lines, then validates <code>pid arrival burst priority</code> .
Input validation	Implemented	Rejects duplicate PIDs, non-positive burst time, negative arrival time, negative priority, extra tokens, malformed records, and embedded NUL bytes.
Built-in samples	Implemented	Provides <code>basic</code> , <code>rr</code> , <code>priority</code> , and <code>edge</code> workloads for quick testing.
Simulation engine	Implemented	Initializes results, appends Gantt entries, merges adjacent entries for the same PID, tracks idle time, and finalizes total runtime.
FCFS	Implemented	Sorts by arrival time and PID, runs each process to completion, and records idle gaps when no process is ready.
SJF	Implemented	Uses a min-heap to select the shortest arrived process in non-preemptive order with deterministic tie-breaking.
Round Robin	Implemented	Uses a FIFO ready queue, configurable quantum, arrival handling during each quantum, requeueing, and context-switch counting.
Priority	Implemented	Uses a min-heap ordered by priority, then arrival and PID. Lower priority number means higher priority.
Metrics	Implemented	Computes waiting time, turnaround time, response time, CPU utilization, throughput, context switches, and total time.
Terminal visualization	Implemented	Prints colored Gantt charts, process metric tables, aggregate summaries, and all-algorithm comparison tables.
CSV export	Implemented	Writes one row per algorithm run with workload summary columns and aggregate metrics.
Browser dashboard	Implemented	Provides a static browser interface with editable processes, sample workloads, algorithm selection, Gantt charts, metric cards, and comparison views.
CSV dashboard generator	Implemented	<code>scripts/visualize_runs.py</code> turns exported CSV rows into a standalone HTML dashboard.
OS process logger	Partial	Samples local processes and maps CPU activity into simulator workloads. This is useful for experimentation, not a claim that the real OS scheduler is reproduced.
C tests	Implemented	Tests cover process behavior, parser validation, FIFO queue, heap ordering, every scheduler, and metrics formulas.

Python helper tests	Implemented	Tests cover CSV dashboard parsing, summary aggregation, HTML report content, and simulator CSV export.
Fuzzing harnesses	Partial	AFL++ parser and queue harnesses exist with seed corpora and dictionary files. Long campaign evidence remains to be added.
Memory validation	Remaining	Valgrind or equivalent sanitizer proof has not yet been recorded in the repository.

4 Updated System Architecture

The system is organized into small C modules with a single command-line entrypoint. The core flow is shown below.



4.1 Module Responsibilities

Module	Responsibility
src/main.c	Parses command-line flags, selects workload source, selects one algorithm or comparison mode, invokes the scheduler, and handles CSV export.

<code>src/input.c</code>	Loads workload files, parses in-memory buffers, skips comments/blanks, and validates process records.
<code>src/process.c</code>	Initializes process structs and resets timing/state fields before each run.
<code>src/queue.c</code>	Implements FIFO queue behavior for Round Robin and min-heap behavior for SJF and priority scheduling.
<code>src/sim_engine.c</code>	Stores the Gantt timeline and computes idle and total runtime.
<code>src/schedulers/*.c</code>	Implements each scheduling policy.
<code>src/metrics.c</code>	Computes process metrics, aggregate metrics, workload summary values, and CSV output.
<code>src/visualization.c</code>	Prints Gantt charts, per-process tables, aggregate metrics, and comparison tables.
<code>dashboard/</code>	Provides a no-build browser dashboard for interactive simulation and visualization.
<code>scripts/</code>	Provides helper scripts for CSV visualization and local OS process sampling.

5 Screenshots and Working Evidence

The repository includes static dashboard screenshots under `stitch_cpu_scheduling_simulator/`. The report also includes terminal output snippets because the C simulator itself is a command-line program.

5.1 Terminal Simulator: All Algorithms

Listing 1: Command: `./cpu_scheduler -s basic -a all`

```
Gantt chart
| P1 0-8 || P2 8-12 || P3 12-21 || P4 21-26 |

Metrics for fcfs
PID ARR BUR PRI START DONE WAIT TURN RESP
  1  0  8  3    0   8   0   8   0
  2  1  4  1    8  12   7  11   7
  3  2  9  4   12  21  10  19  10
  4  3  5  2   21  26  18  23  18
avg_wait=8.75 avg_turn=15.25 avg_resp=8.75 util=1.00 thr=0.15 ctx=0 total=26

Comparison
algo      wait      turn      resp      util      thr      ctx
fcfs      8.75      15.25     8.75      1.00      0.15      0
sjf       7.75      14.25     7.75      1.00      0.15      0
rr        12.75     19.25     2.00      1.00      0.15     12
priority   7.75      14.25     7.75      1.00      0.15      0
```

5.2 Terminal Simulator: Round Robin

Listing 2: Command: `./cpu_scheduler -f workloads/sample_rr.txt -a rr -q 4`

```
Gantt chart
| P1 0-4 || P2 4-7 || P3 7-10 || P1 10-30 |
Ticks: 0 5 10 15 20 25 30

Metrics for rr
PID ARR BUR PRI START DONE WAIT TURN RESP
  1  0  24  0    0  30   6  30   0
  2  0  3  0    4   7   4   7   4
  3  0  3  0    7  10   7  10   7
avg_wait=5.67 avg_turn=15.67 avg_resp=3.67 util=1.00 thr=0.10 ctx=3 total=30
```

5.3 Browser Dashboard Screenshot

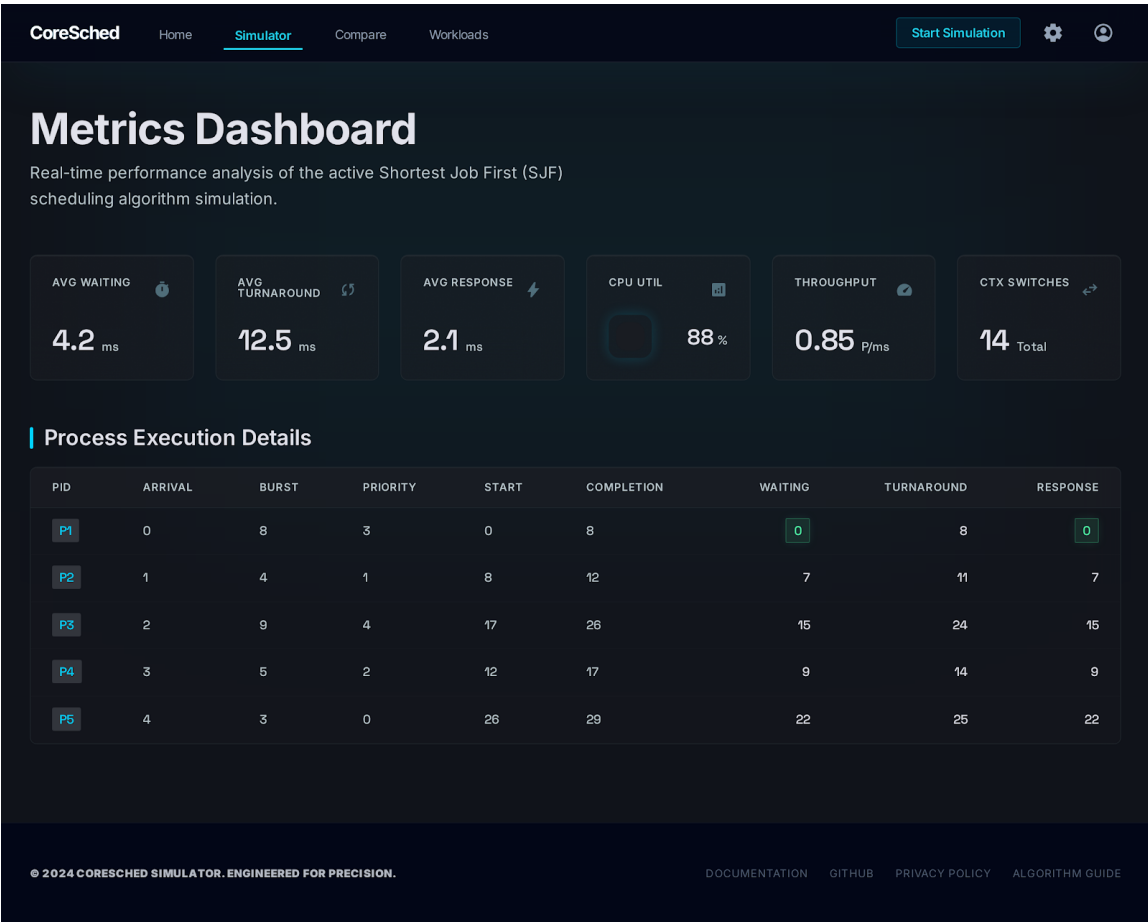


Figure 1: Metrics dashboard design for comparing scheduler output.

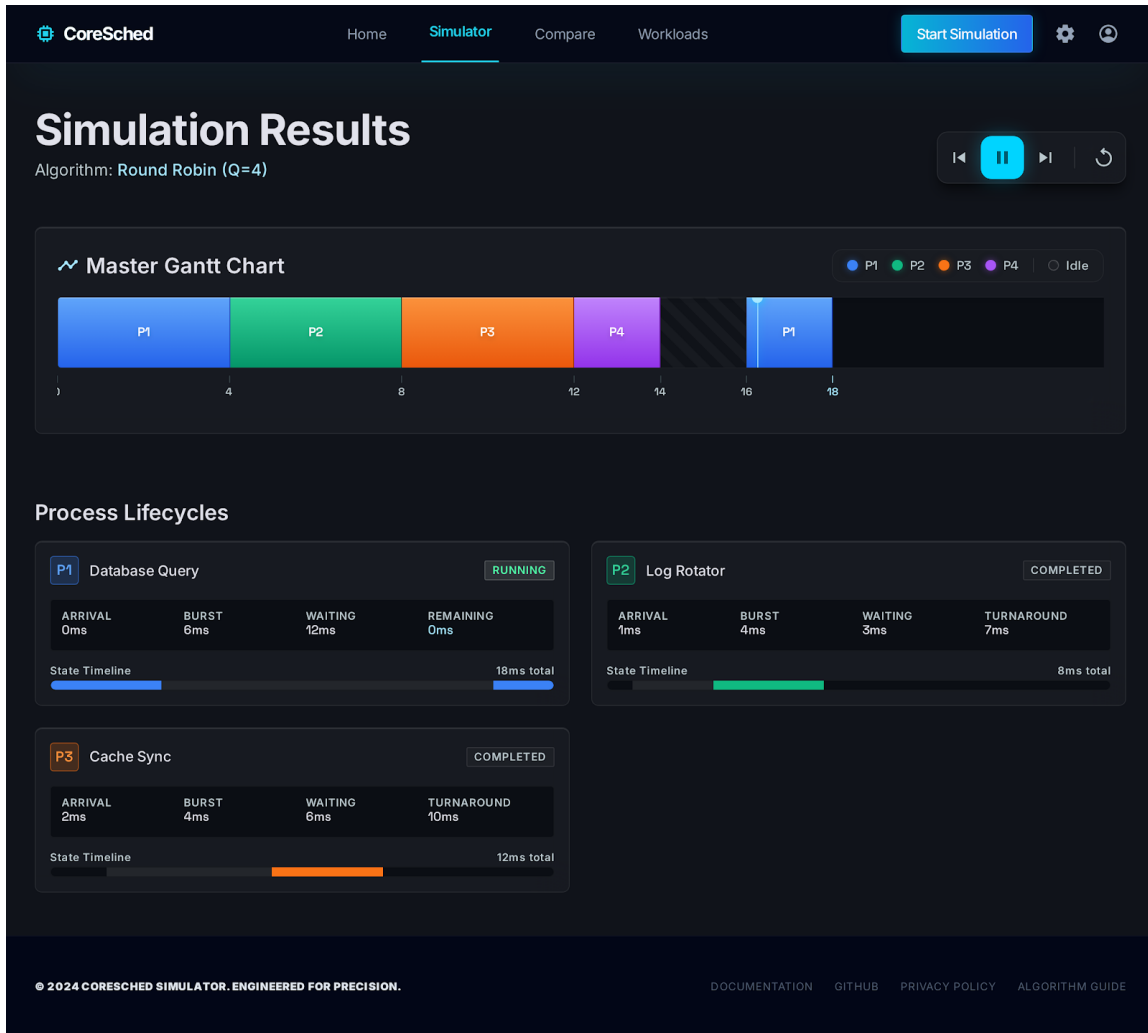


Figure 2: Gantt chart visualization for scheduler timeline output.

5.4 Test Evidence

Listing 3: Validated command: make test

```
test_process: PASS 19/19 passed
test_queue: PASS 22/22 passed
test_input: PASS 21/21 passed
test_fcfs: PASS 19/19 passed
test_sjf: PASS 12/12 passed
test_rr: PASS 9/9 passed
test_priority: PASS 8/8 passed
test_metrics: PASS 13/13 passed
pytest tests_py: 7 passed
```

6 Challenges Faced

- **Consistent process timing:** Each algorithm updates start time, completion time, response time, and remaining time differently. A shared reset path was needed so comparison mode could run algorithms back-to-back.
- **Deterministic tie-breaking:** Scheduling algorithms can produce ambiguous results when several processes arrive together or share the same burst/priority. The implementation uses stable tie-breakers so tests and reports are reproducible.
- **Round Robin queue order:** Round Robin must add processes that arrive during a time slice and then decide whether to requeue the current process. That ordering affects response time, waiting time, and context switch counts.
- **Input validation:** The parser must handle normal workloads and malformed inputs without crashing. This required explicit rejection of bad integer fields, duplicate PIDs, invalid burst values, extra tokens, and NUL bytes.
- **Metric correctness:** Waiting time, turnaround time, and response time are easy to mix up. Tests were added around the formulas to keep output consistent.
- **Scope control:** A previous experimental prediction layer made the project look less focused. It has now been removed so the repository directly supports the Operating Systems Phase 2 requirements.
- **Validation gap:** Unit tests cover the main behavior, but they are not a proof of correctness for every possible workload. Fuzzing and memory-check evidence still need to be run and documented.

7 Plan for Completing Remaining Work

1. Compile this LaTeX file into the final Phase 2 PDF and include it with the code submission.
2. Run ‘make clean’, ‘make’, ‘make test-c’, and Python helper tests in the target grading environment, then record the exact output.
3. Add more tests for large workloads, maximum process count boundaries, context-switch overhead, CSV comparison output, and invalid command-line arguments.
4. Standardize context-switch overhead across all algorithms. Currently FCFS and Round Robin consider overhead more directly than SJF and priority.
5. Run Valgrind or an equivalent sanitizer pass and add the result to the final report.
6. Run AFL++ parser and queue campaigns for a documented duration, then save summary stats and any minimized reproducers.
7. Capture final screenshots from the actual dashboard and terminal output if the submission platform requires image files instead of report snippets.
8. Review all source comments for clarity and add comments only where the scheduling or validation logic is not obvious.

8 Code Submission Organization

The repository is organized as follows:

include/	Public C headers
src/	C simulator implementation
src/schedulers/	FCFS, SJF, Round Robin, and priority schedulers
tests/	C unit tests
tests_py/	Python tests for helper scripts and CSV export
workloads/	Sample scheduler workloads
dashboard/	Static browser dashboard
scripts/	CSV dashboard generator and process logger
fuzz/	AFL++ harnesses, dictionaries, and seed corpora
reports/	Phase report source
Makefile	Build, test, and fuzz commands
README.md	Grader-facing instructions
DASHBOARD_SPEC.md	Dashboard feature and design specification

8.1 Main C Files

File	Purpose
include/process.h src/process.c	and Process data model, initialization, reset, and state labels.
include/input.h src/input.c	and Workload loading, in-memory parsing, built-in samples, and validation.
include/queue.h src/queue.c	and FIFO queue and min-heap support structures used by schedulers.
include/scheduler.h src/schedulers/*.c	and Scheduler dispatch and algorithm implementations.
include/sim_engine.h src/sim_engine.c	and Gantt timeline storage, idle time, and total runtime finalization.
include/metrics.h src/metrics.c	and Metric formulas, process table output, workload summaries, and CSV export.
include/visualization.h and src/visualization.c	Terminal Gantt chart, metric summary, and comparison table rendering.
src/main.c	CLI option parsing, workload selection, algorithm execution, comparison mode, and CSV output.

8.2 Audit Result

The final Phase 2 cleanup removed the underlying prediction code and remote inference support from the repository. The active source tree no longer contains the former model training files, prediction server, or model artifact. The command-line interface also no longer exposes the old prediction flag.

The remaining Python code is helper tooling for this OS project:

- `scripts/visualize_runs.py`: builds HTML reports from simulator CSV output.
- `scripts/os_process_logger.py`: generates experimental simulator workloads from local process CPU samples.
- `tests_py/`: verifies helper script behavior and CSV export.

9 Conclusion

Phase 2 now contains a working implementation of the core CPU scheduling simulator. The project has the main scheduling algorithms, input validation, timeline generation, metrics, terminal output, CSV export, browser visualization, helper scripts, and tests. The remaining work is mostly final validation and submission polish rather than core implementation.